NUMERICAL LINEAR ALGEBRA

Sparse Direct Solvers Algorithms and Code

Sparse LU factorization, structural analysis,
and a Python implementation inspired by SuperLU.



Sparse matrix A Sparse factors L, U

An introduction to the mathematics, the solver pipeline, and the implementation. All figures are editable TikZ.

The task is to solve a sparse linear system accurately

$$Ax = b, \qquad A \in \mathbb{R}^{n \times n}, \quad \mathrm{nnz}(A) \ll n^2.$$

Sparse direct approach

- Factor A into triangular factors, with permutations as needed.
- Solve with those factors for one or many right-hand sides.
- Use residuals and error estimates to assess the result.

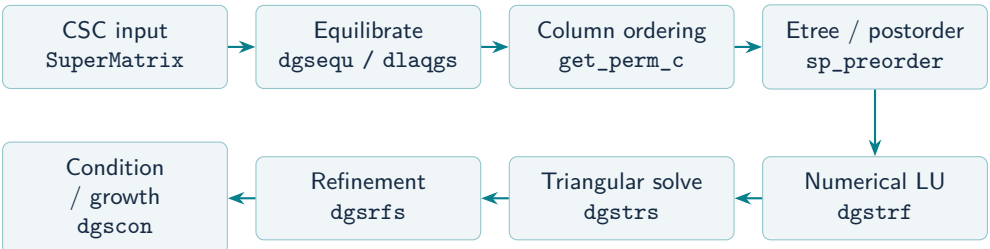
What determines the cost?

- Input nonzeros and their graph structure.
- Fill-in created during factorization.
- Pivoting, ordering, and block organization.

Factorization
expensive, reusable work → **Triangular solves**
apply factors to each RHS → **Verification**
residual and conditioning

Key idea: The code implements real-valued sparse LU. It supports nonsymmetric matrices; nonsingularity is required for a unique solution.

SuperLU's driver separates preparation, factorization, and solve

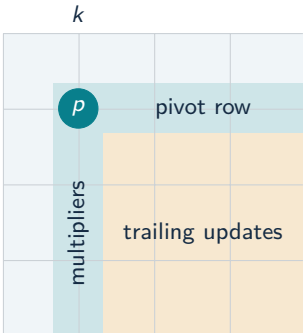


This is a learning sequence. The expert driver can compute condition and pivot-growth diagnostics before solving.

Source guide: `superlu/SRC/dgssvx.c`

Key idea: Use the driver to learn the stage boundaries before reading the inner numerical kernels.

A scalar LU step stores a multiplier and updates a row



For each row $i > k$ containing column k:

$$l_{ik} = \frac{a_{ik}}{u_{kk}}$$

$$a_{ij} \leftarrow a_{ij} - l_{ik} u_{kj}, \qquad j > k.$$

- Store l_{ik} in L.
- Remove the eliminated entry from the working row.
- Insert new entries when the update creates fill.

Key idea: The division creates L; the row updates create U and the evolving Schur complement.

Eliminating one pivot can turn zeros into nonzeros

Before the first elimination

$$\begin{bmatrix} 4 & 1 & 1 \\ 1 & 4 & 0 \\ 1 & 0 & 4 \end{bmatrix}$$

$$\longrightarrow \quad l_{10} = l_{20} = 1/4$$

Trailing block after elimination

$$\begin{bmatrix} 4 & 1 & 1 \\ 0 & 15/4 & -1/4 \\ 0 & -1/4 & 15/4 \end{bmatrix}$$

$$a_{12}^{\text{new}} = 0 - \left(\frac{1}{4}\right)(1) = -\frac{1}{4}, \qquad a_{21}^{\text{new}} = -\frac{1}{4}.$$

Key idea: These two orange entries are fill-in. Sparse input does not guarantee equally sparse factors.

Partial pivoting selects a usable pivot row

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 0 \\ 2 & 0 & 3 \end{bmatrix}$$

The first diagonal is zero.

Swap rows 0 and 2

$$\begin{bmatrix} 2 & 0 & 3 \\ 1 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix}$$

Eliminate column 0

$$\begin{bmatrix} 2 & 0 & 3 \\ 0 & 1 & -3/2 \\ 0 & 2 & 1 \end{bmatrix}$$

$$p = 2, \quad l_{10} = \frac{1}{2}, \quad a_{12}^{\text{new}} = 0 - \frac{1}{2}(3) = -\frac{3}{2}.$$

Key idea: Full partial pivoting chooses the largest magnitude in the active column. Threshold pivoting can prefer a sufficiently large original diagonal.

A late row swap must also move previous multipliers

Before the second pivot

$$W = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 1 & -3/2 \\ 0 & 2 & 1 \end{bmatrix}$$

Previous L column: $\begin{bmatrix} 1 & 1/2 & 0 \end{bmatrix}^T$



Swap active rows 1 and 2

$$W = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 2 & 1 \\ 0 & 1 & -3/2 \end{bmatrix}$$

Previous L column: $\begin{bmatrix} 1 & 0 & 1/2 \end{bmatrix}^T$

```
U[k], U[p] = U[p], U[k]
L[k], L[p] = L[p], L[k]
perm[k], perm[p] = perm[p], perm[k]
```

Key idea: The old multiplier belongs to an equation row. Moving that row must move its elimination history.

The finished factors satisfy $PA = LU$

$$PA = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 2 & 1 \\ 1 & 1 & 0 \end{bmatrix}, \qquad \texttt{perm} = [2, 0, 1].$$

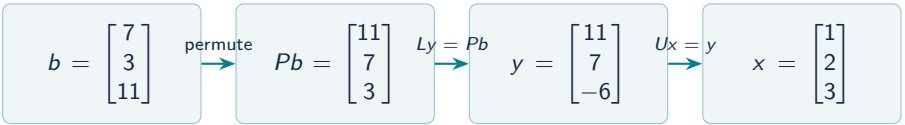
$$L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1/2 & 1/2 & 1 \end{bmatrix}, \qquad U = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 2 & 1 \\ 0 & 0 & -2 \end{bmatrix}.$$

The diagonal of L is one. The teaching code stores it implicitly.

`perm[i]` is the original row now occupying factor row i.

Key idea: Check the factorization identity independently. Correct-looking triangular factors are insufficient if the permutation is wrong.

Solving uses the factors, not another elimination



Forward substitution

$$y_i = (Pb)_i - \sum_{j<i} l_{ij} y_j.$$

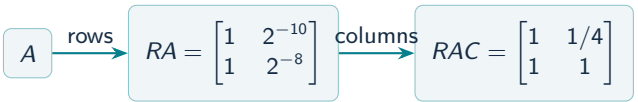
Backward substitution

$$x_i = \frac{y_i - \sum_{j>i} u_{ij} x_j}{u_{ii}}.$$

Key idea: Once L, U, P exist, multiple right-hand sides reuse the same factorization.

Equilibration balances row and column magnitudes

$$A = \begin{bmatrix} 2^{-10} & 2^{-20} \\ 2^{10} & 2^{2} \end{bmatrix}, \quad R = \operatorname{diag}(2^{10}, 2^{-10}), \quad C = \operatorname{diag}(1, 2^{8}).$$



Candidate row scales are inverse row maxima. Candidate column scales use the row-scaled matrix. Automatic selection uses scale ratios and safe magnitude limits; it need not apply both.

Source guide: `dgsequ.c; dlaqgs.c`

Key idea: Equilibration changes the numerical representation of the same linear problem. It does not remove singularity.

Scaling and column ordering add two more maps

$$\boxed{P_r\,(RAC)\,P_c = LU}$$

Normal solve $Ax = b$

$$\begin{aligned} \widehat{b} &= P_r R b, \\ Ly &= \widehat{b}, \\ Uz &= y, \\ x &= C P_c z. \end{aligned}$$

Conventions in the Python code

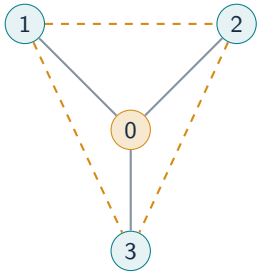
- R, C: positive diagonal scales.
- p_k: original row at factor row k.
- q_k: original column at factor column k.
- $\widehat{b}_k = R_{p_k p_k} b_{p_k}$.
- $x_{q_k} = C_{q_k q_k} z_k$.

SuperLU's `perm_r`/`perm_c` arrays use the inverse, old-to-new convention.

Key idea: Row permutations reorder equations; column permutations reorder unknowns. Scaling must also be undone in the solution.

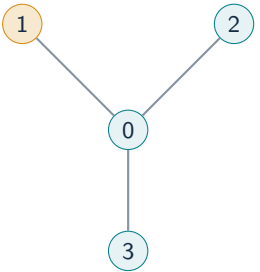
Eliminating a high-degree vertex can create many fill edges

Eliminate the center first



Its neighbors become a clique.

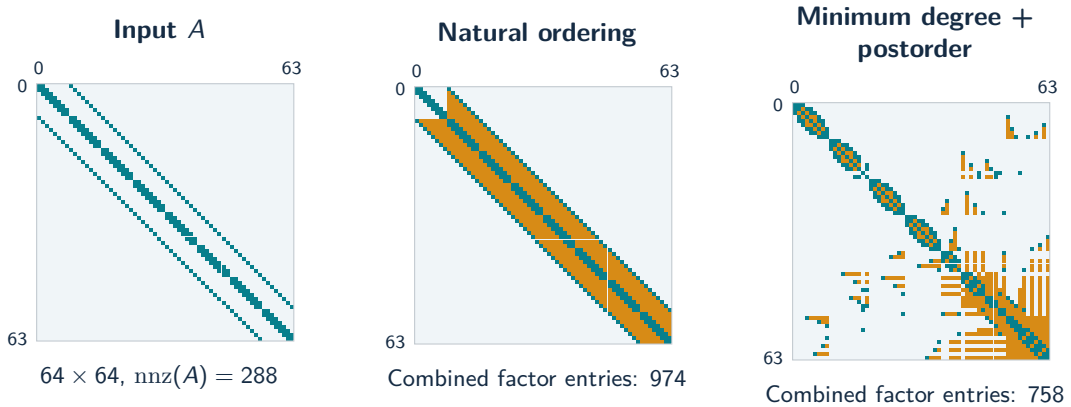
Eliminate a leaf first



A single neighbor needs no fill edge.

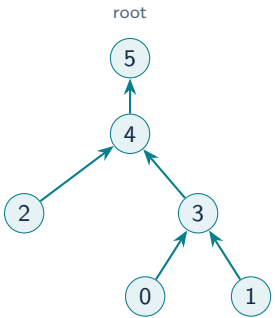
Key idea: This graph-elimination illustration motivates minimum-degree ordering. Our code applies it to the structural column-intersection graph of $A^T A$.

Ordering reduces factor storage on the same Poisson problem



Key idea: Plots come from the Python factors. Combined counts are $\mathrm{nnz}(L) + \mathrm{nnz}(U) - n$, counting the diagonal once.

An elimination tree organizes structural dependencies



Illustrative parent array: $[3, 3, 4, 4, 5, 6]$.
Sentinel 6 denotes a root.

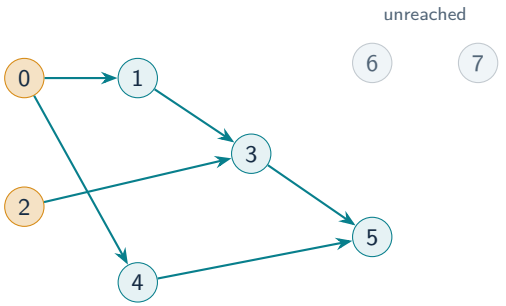
The preprocessing tree

- SuperLU normally builds the column etree of the structural $(AP_c)^T(AP_c)$.
- No numerical normal equations are solved.
- A child-before-parent postorder groups complete subtrees contiguously.
- Small subtrees can become relaxed supernodes.

One postorder here is $[2, 0, 1, 3, 4, 5]$.

Key idea: The tree is structural guidance. For general LU, numerical pivoting still affects the actual factor structure.

DFS reach finds which previous factor columns can contribute



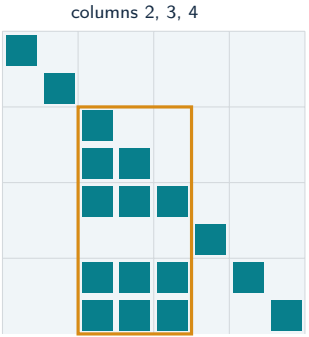
Scalar triangular-update illustration

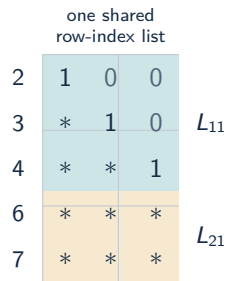
- An edge $j \to i$ represents a nonzero l_{ij}.
- Start from sparse input support: nodes 0 and 2.
- Reach follows dependencies; update in topological order.
- Nodes 6 and 7 need no update in this example.

SuperLU groups reached contributions into supernodal segments and accounts for pivoted row indices.

Key idea: SuperLU's symbolic DFS is interleaved with numerical factorization. Our graph-envelope preprocessing is a simpler teaching substitute.

A supernode shares row indices across consecutive columns





Key idea: The common rows below the diagonal block are 6 and 7. Packed dense blocks can store zeros; their triangular structure still matters. SuperLU forms supernodes during factorization.

Our Python panels perform a genuine block Schur update



After accounting for panel row swaps:

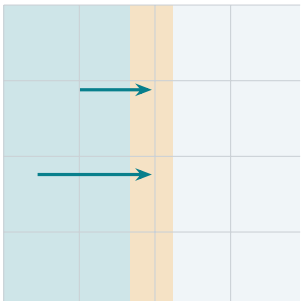
$$
\begin{aligned}
A_{11} &= L_{11}U_{11}, \\
L_{11}U_{12} &= A_{12}, \\
L_{21}U_{11} &= A_{21}, \\
A_{22}^{\text{new}} &= A_{22} - L_{21}U_{12}.
\end{aligned}
$$

The code computes L_{21} while factoring panel columns, delays trailing updates, solves for U_{12}, then updates touched trailing positions.

Key idea: Block organization exposes dense arithmetic. Native optimized solvers use tuned kernels; this Python code keeps the operations explicit for study.

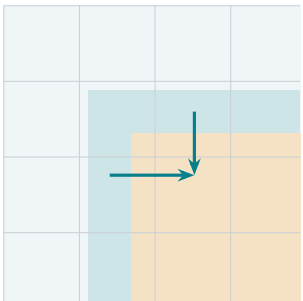
Left-looking and right-looking schedule the same algebra differently

SuperLU: left-looking



Gather earlier contributions into the current column/panel.

Our panel kernel: right-looking



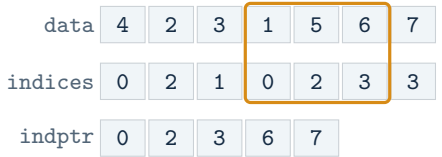
Push the finished panel's contribution into the trailing matrix.

Key idea: A similar stage diagram does not imply the same implementation. Scheduling changes workspace, locality, and sparse traversal.

CSR stores values and column indices, row by row

$$A = \begin{bmatrix} 4 & 0 & 2 & 0 \\ 0 & 3 & 0 & 0 \\ 1 & 0 & 5 & 6 \\ 0 & 0 & 0 & 7 \end{bmatrix}$$

Highlighted: row $i = 2$ (zero-based indexing).



$\texttt{indptr}[2] = 3, \quad \texttt{indptr}[3] = 6$

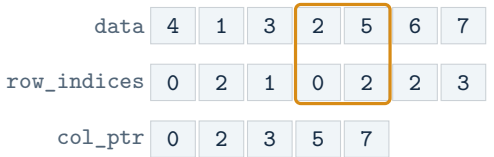
Read storage positions $k = 3, 4, 5$.

Key idea: i is a matrix row, k is a storage position, and $j = \texttt{indices}[k]$ is a matrix column.

CSC groups the same entries by column

$$A = \begin{bmatrix} 4 & 0 & 2 & 0 \\ 0 & 3 & 0 & 0 \\ 1 & 0 & 5 & 6 \\ 0 & 0 & 0 & 7 \end{bmatrix}$$

Highlighted: column $j = 2$.

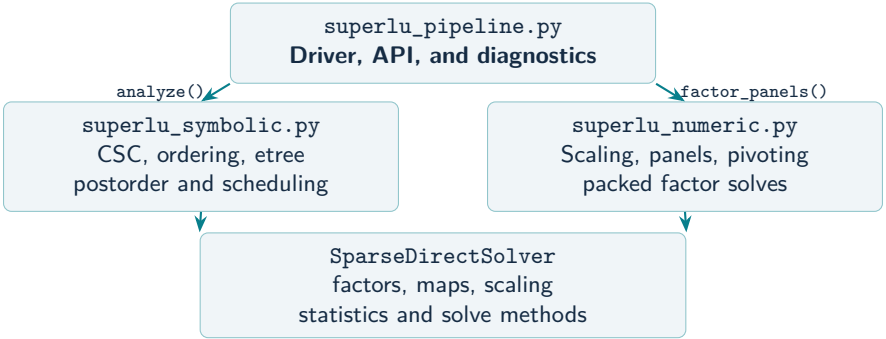


`col_ptr`$[2] = 3$, `col_ptr`$[3] = 5$

Column 2 has entries at rows 0 and 2.

Key idea: SuperLU processes CSC input. The Python implementation provides a CSC view for structure and diagnostics, while dictionary rows carry the numerical working matrix.

The code separates analysis, factors, and solves



Supporting code: `sparse_solver.py` supplies CSR and Poisson construction; `sparse_lu.py` provides a compact scalar LU baseline.

Key idea: A structural plan is reusable across value changes. Numerical factors are reusable across RHS changes. The driver owns both lifecycles.

The public API follows analyze, factorize, solve, and assess

```python
from sparse_learning.sparse_solver import poisson_2d
from sparse_learning.superlu_symbolic import analyze
from sparse_learning.superlu_pipeline import factorize

A = poisson_2d(8)
b = A.matvec([1.0] * 64)
plan = analyze(A, ordering="minimum_degree")
solver = factorize(A, analysis=plan)
x = solver.solve(b)
report = solver.refine(b, x)

print(report.componentwise_backward_error)
```

Objects you can inspect

- `plan`: ordering, etree, patterns, panels.
- `solver`: L, U, permutations, scales, supernodes.
- `report`: solution, residual, error and condition estimates.

Key idea: Calling `factorize(A)` directly performs analysis automatically. Passing a plan makes structural reuse explicit.

The symbolic module turns sparsity into an execution plan

```
graph = column_graph(A)
order = minimum_degree(graph)
etree, columns = symbolic_cholesky(
    permute_graph(graph, order))

post = tree_postorder(etree)
order = [order[j] for j in post]
etree, columns = symbolic_cholesky(
    permute_graph(graph, order))

relaxed = relaxed_supernodes(etree, relax)
panels = panel_schedule(n, panel_size, relaxed)
```

Excerpt of the default minimum-degree / postorder path in `analyze()`.

Stored in `SymbolicAnalysis`

- Canonical input pattern.
- New-to-old column order.
- Etree and structural column sets.
- Strict / relaxed groups and panel ranges.

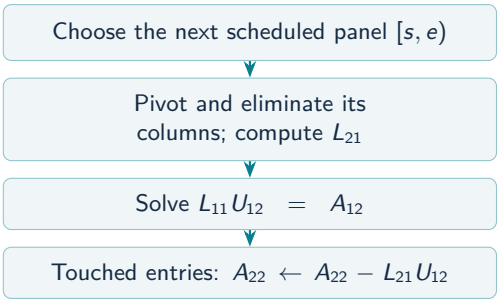
Pattern checks prevent reuse after structural changes.

Key idea: This implementation models the structural $A^T A$ envelope. Its column sets guide scheduling rather than preallocating exact pivoted LU patterns.

The numerical module maintains a sparse Schur complement

State in `factor_panels()`

- `work`: completed U rows and the active Schur complement.
- `lower`: previously computed multipliers.
- `rows / row_position`: equation permutation and its inverse.
- `stats`: swaps, fill insertions, work size.



Key idea: The numerical state grows when fill appears. Exact zeros are removed; small nonzero entries are retained. Row swaps also move existing multipliers.

The solve method applies maps around a packed triangular solve

```
q, p = self.analysis.col_order, self.row_order
rhs = [self.scaling.row[i] * b[i] for i in p]
z = triangular_solve(self.supernodes, rhs)
x = [0.0] * self.n
for k, old in enumerate(q):
    x[old] = self.scaling.column[old] * z[k]
```

Normal-solve path from `SparseDirectSolver.solve()`.

Packed factor storage

- Each `Supernode` holds $L_{11}, U_{11}, L_{21}, U_{12}$ and shared indices.
- `U_csc` exposes an additional CSC view for inspection.

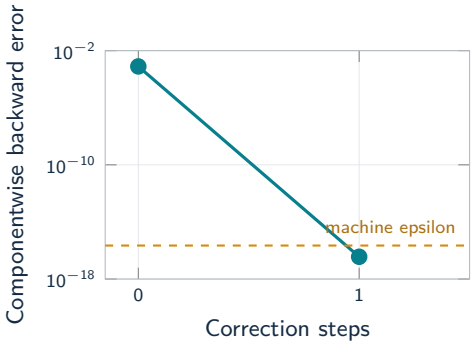
Transpose solve $A^T x = b$

$$\begin{aligned} \widehat{b} &= P_c^T C b, \\ U^T v &= \widehat{b}, \\ L^T z &= v, \\ x &= R P_r^T z. \end{aligned}$$

Transpose solves also support inverse-norm estimation.

Key idea: Permutations and scaling are part of the solve algorithm. The factor-space solution must be mapped back to the original variable order.

Refinement uses residuals to correct the computed solution



Actual scaled 8×8 grid; initial solution perturbed by 10^{-3}.
80-digit residual arithmetic, float64 factors.

$$r = b - Ax,$$
$$A\,\delta x = r \quad \text{(reuse LU)},$$
$$x \leftarrow x + \delta x.$$

Diagnostics answer different questions

$$\mathrm{berr} = \max_i \frac{|r_i|}{(|A||x| + |b|)_i}$$

$$\mathrm{rcond}_1 \approx \frac{1}{\|A\|_1 \|A^{-1}\|_1}.$$

Condition and forward-error estimates use normal and transpose solves without forming an inverse.

Key idea: A small residual alone does not guarantee small solution error. Conditioning connects those two quantities.

Reuse depends on what changed

Change	What can be reused?	What must run again?
Only b changes	Scaling, order, symbolic plan, L, U	Triangular solves; optional refinement
Values of A change; pattern stays	Column order, etree, scheduling plan	Numerical factorization; scaling can be recomputed
Sparsity pattern changes	Existing plan needs revalidation	Ordering and symbolic analysis, then factorization

`solver.solve_many(rhs_columns)`
reuse numerical factors

`solver.refactor(new_A)`
reuse the structural plan

Key idea: An old pivot sequence is only a preference. The Python refactorization checks each reused pivot against the current threshold.

Verification checks algebra, numerical accuracy, and reuse

Stage and integration checks

- Factor identity $P_r R A C P_c = LU$.
- Initial and late pivoting; fill creation.
- Normal / transpose solves and scaling modes.
- Supernode blocks and panel widths.
- Multiple RHS, same-pattern reuse, singular inputs.

Independent SuperLU comparison

34 tests passed
Stage-level and end-to-end checks

312 solve comparisons passed
156 configurations, normal + transpose

Largest relative solution difference: 2.9×10^{-15}.
Checks use SciPy's actual SuperLU implementation as an independent reference.

Key idea: Tests establish behavior on the checked matrices; they do not establish production performance or coverage of every matrix class.

The actual SuperLU loop interleaves symbolic and numeric work

At the panel level

`dpanel_dfs`
symbolic reach for the panel

↓

`dpanel_bmod`
apply earlier supernode updates

Then process each column inside the panel.

For each column

`dcolumn_dfs`: reach + supernode boundary

↓

`dcolumn_bmod`: numerical updates

↓

`dcopy_to_ucol`: store completed U segments

↓

`dpivotL`: choose pivot + compute multipliers

↓

`dpruneL`: reduce later symbolic traversal

Key idea: Read the main loop in `dgstrf.c`, then follow these calls individually. Relaxed-supernode processing provides a separate path in that loop.

The teaching pipeline and SuperLU share stages, not all kernels

Aspect	This Python implementation	Local SuperLU source
Input / work	CSR input, CSC view, mutable dictionary work	CSC input, specialized sparse arrays and panel workspace
Ordering	Exact minimum degree on structural $A^T A$	COLAMD, minimum-degree variants, other options
Symbolic work	Explicit graph fill / etree envelope	Panel and column DFS, reach segments, pruning
Numerical kernel	Blocked right-looking panels	Left-looking supernodal updates
Supernodes	Detected and packed after numerical LU; used by solves	Formed during factorization, including relaxed groups
Shared roles	Scaling, permutations, pivot tests, solves, refinement, reuse	Corresponding driver stages

Key idea: Use the Python implementation to inspect the algebra. Use the source call graph to learn SuperLU's specific storage and traversal strategy.

Run the code and connect each stage to its source

Trace a complete factorization

```
python3 sparse_learning/superlu_pipeline.py \
    --grid 8 --trace

python3 sparse_learning/superlu_pipeline.py \
    --example scaled --grid 8

python3 -m unittest discover -s sparse_learning
```

The solver and deck figure generator require only the Python standard library. Independent reference validation additionally uses NumPy and SciPy.

SuperLU source entry points

- `dgssv.c`: basic driver.
- `sp_preorder.c`: column maps and etree.
- `dgstrf.c`: panel / column factorization loop.
- `dgstrs.c`: factor solves.
- `dgssvx.c`: expert driver and diagnostics.

Key idea: Follow the same progression in mathematics and code: maps, structure, numerical factors, triangular solves, and accuracy checks.